UFC FIGHT MATCHMAKER

Team Sprint Plan — 48 Hours

Machine Learning Course Project

Team Members

Raji — Lead — Model Architecture, Baselines, SHAP, Integration

Lorenzo — Data — Ground-Truth Labels, Scraping, Backtesting

Gustave — Features — Cross-Features, Dead Code Fixes, Dashboard

Mattheus — Evaluation — Temporal Split, CV, Calibration, Slides

Deadline: 48 hours from kickoff

Executive Summary

This document assigns every task across the 48-hour sprint to a specific team member with step-by-step implementation instructions. The goal is to transform the current prototype into a scientifically rigorous, presentation-ready system.

The Core Problem We Are Fixing

The current neural network is trained to predict its own heuristic score — a hand-coded formula the team invented. This is circular: the model cannot learn anything that the formula does not already capture. There is no external ground truth. We are fixing this by introducing real-world fight quality labels (Fight of the Night bonuses) and applying proper ML methodology throughout.

Key Changes (All Hands)

1. Real labels: Replace the invented 0–100 score with actual UFC Fight of the Night / Performance of the Night bonus data (binary classification).

2. Cross-features: Activate the 24 matchup cross-features that are fully implemented but never concatenated into the input vector (48 → 72 features).

3. Temporal split: Split train/val/test by date (not random) to prevent data leakage. Ensure symmetric augmentation pairs stay in the same fold.

4. Baselines: Compare the NN against XGBoost, Random Forest, and Logistic Regression on the same splits. If a simpler model wins, that is a valid and honest finding.

5. Interpretability: SHAP values, feature importance, calibration plots, and Spearman rank correlation.

6. Hyperparameter tuning: GridSearchCV / RandomizedSearchCV on the tree-based models; structured sweep on the NN.

Timeline Overview

	Day 1 (Hours 0–12)	Day 2 (Hours 12–24+)
Raji	XGBoost + RF baselines, SHAP setup, oversee integration	NN hyperparameter sweep, final integration, presentation review
Lorenzo	Scrape FOTN/POTN data, merge into DB, create labels column	Backtesting analysis, help with presentation content
Gustave	Activate cross-features, fix input_dim, fix all dead code	Update dashboard to reflect new model, clean CLI

Mattheus	Temporal split, fix augmentation leakage, k-fold CV	Calibration plots, Spearman's rho, build slides

RAJI — Lead — Model Architecture & Baselines

You own the modeling pipeline end-to-end. Your job is to build the baseline comparisons, run SHAP analysis, tune hyperparameters, and integrate everyone else's work into a coherent system.

Day 1 Tasks

Task R1: XGBoost Baseline

Once Lorenzo's labels are merged and Gustave's cross-features are active, train an XGBoost classifier on the 72-feature vector with the binary FOTN label. Use Mattheus's temporal split.

[CRITICAL] Implementation Steps

- Create models/baselines.py with a class BaselineComparison
- Load the dataset with the new binary labels (1 = bonus fight, 0 = not)
- Use Mattheus's temporal split (train: pre-2023, val: 2023, test: 2024+)
- Train XGBClassifier with: n_estimators=300, max_depth=6, learning_rate=0.05, scale_pos_weight=(count_neg/count_pos)
- scale_pos_weight handles class imbalance — FOTN fights are rare (~10-15% of cards)
- Run RandomizedSearchCV (n_iter=50, cv=TimeSeriesSplit(n_splits=5)) over: max_depth [3,5,7,9], learning_rate [0.01,0.05,0.1], subsample [0.7,0.8,0.9], colsample_bytree [0.7,0.8,0.9], min_child_weight [1,3,5]
- Store best params and best CV score

Task R2: Random Forest Baseline

Same split, same features, different model family. This tests whether bagging outperforms boosting on this data.

[HIGH] Implementation Steps

- Train RandomForestClassifier with n_estimators=500, class_weight='balanced'
- Run RandomizedSearchCV (n_iter=30) over: n_estimators [200,500,1000], max_depth [5,10,15,None], min_samples_split [2,5,10], min_samples_leaf [1,2,4], max_features ['sqrt','log2']
- Also train a LogisticRegression(class_weight='balanced') as a linear baseline — takes 2 minutes, gives a floor to beat

Task R3: SHAP Analysis

You've done this on the SMR project. Same workflow here — SHAP on XGBoost is the easiest and most interpretable.

[HIGH] Implementation Steps

- pip install shap
- Create models/interpretability.py
- Use shap.TreeExplainer(xgb_model) for XGBoost (fast, exact)
- Generate: shap.summary_plot (beeswarm), shap.plots.bar (global importance), shap.force_plot for 2-3 specific matchup predictions
- Save all plots as PNGs in outputs/ for the presentation
- Sanity check: if a weird feature (e.g., height_cm) dominates importance, something is wrong upstream
- For the NN: use shap.DeepExplainer or shap.GradientExplainer — compare feature importance rankings between NN and XGBoost

Day 2 Tasks

Task R4: Neural Network Hyperparameter Sweep

Now that the NN has real labels and proper features, tune it systematically.

[CRITICAL] Implementation Steps

- Switch the NN output from sigmoid (regression) to binary classification: single output with BCEWithLogitsLoss
- Sweep over: hidden_dims [(128,64,32), (256,128,64), (64,32)], dropout [0.2,0.3,0.5], learning_rate [1e-3, 5e-4, 1e-4], weight_decay [1e-4, 1e-3], batch_size [32,64]
- Use the validation set (2023 fights) for model selection, NEVER touch the test set until final evaluation
- Track: F1 (primary), AUC-ROC, precision, recall for each config
- If the NN doesn't beat XGBoost — that is a VALID finding. Document it honestly. Professors respect intellectual honesty over forcing a deep learning narrative.

Task R5: Final Model Comparison Table

Produce the comparison table that goes in the presentation.

[HIGH] Implementation Steps

- Evaluate ALL models on the held-out TEST set (2024+ fights): Logistic Regression, Random Forest, XGBoost, Neural Network
- Metrics for each: Accuracy, Precision, Recall, F1, AUC-ROC, Spearman's rank correlation
- Create a clean comparison DataFrame, save as CSV and as a formatted table image
- Write a paragraph interpreting the results: which model won, why, what features mattered most

Task R6: Integration

You are the integration point. Make sure everyone's code merges cleanly.

[CRITICAL] Implementation Steps

- Create a branch 'sprint-integration' and merge Lorenzo's labels, Gustave's features, Mattheus's splits
- Run the full pipeline end-to-end: data → features → split → train → evaluate
- Resolve any dimension mismatches (input_dim must match actual feature count)
- Final smoke test: predict matchups for a current UFC card and verify outputs look sane

LORENZO — Data — Ground-Truth Labels & Backtesting

You own the single most important fix in this project: replacing the fake heuristic training target with real-world UFC bonus data. Without your work, the model is scientifically meaningless. Everything depends on you delivering clean labels by end of Day 1.

Day 1 Tasks

Task L1: Scrape FOTN/POTN Bonus Data

The UFC awards Fight of the Night and Performance of the Night bonuses at every event. This is our ground-truth label for fight quality.

[CRITICAL] Implementation Steps

- Target source: UFCStats.com event pages — bonus winners are listed on each event page
- Alternative source: Wikipedia UFC event pages list bonus winners in a consistent table format
- Add a new scraper function in scrapers/ufc_stats_scraper.py: scrape_event_bonuses(event_url)
- For each event, extract: event_name, event_date, bonus_type (FOTN / POTN), fighter_name(s)
- Store in a new table in the SQLite DB: fight_bonuses (event_id, fight_id, bonus_type)
- Match bonus winners to existing fight records using fighter names + event — handle name mismatches carefully
- Expected yield: ~500-800 labeled fights across all UFC events since bonuses were introduced (~2005)

Task L2: Create Binary Label Column

Transform the scraped bonus data into the training label.

[CRITICAL] Implementation Steps

- Add a column to the fights table or create a view: is_bonus_fight (1 if FOTN or POTN, 0 otherwise)
- FOTN is most relevant (labels the fight itself), POTN labels a fighter's individual performance — include both
- Verify class distribution: expect ~10-15% positive class (bonus fights). Print the counts.
- Update feature_engineering.py: build_training_dataset() should return (X, y) where y is the binary label, NOT the old compute_fight_quality_score
- Remove or comment out compute_fight_quality_score — do NOT delete it, just stop using it as the target

Task L3: Data Quality Checks

[HIGH] Implementation Steps

- Print total fights in DB, total with bonus labels, total without — verify no major gaps
- Check for duplicate bonus entries (same fight labeled twice)
- Verify temporal coverage: bonuses should span 2005–present, flag any missing years
- Save a summary CSV: event_name, date, num_fights, num_bonus_fights for quick review

Day 2 Tasks

Task L4: Backtesting Analysis

This is the “wow factor” for the presentation. Take past UFC cards and check if the model would have predicted the actual bonus fights.

[HIGH] Implementation Steps

- Select 5–10 recent UFC events from the test set (2024+)
- For each event, run the trained model on all fights and rank them by predicted probability
- Compare the model’s top predictions to the actual FOTN/POTN winners
- Compute hit rate: what % of actual bonus fights were in the model’s top-3 predictions per card?
- Create a clean table for the presentation: Event | Model’s Top Pick | Actual FOTN | Hit?
- Even a 30-40% hit rate is impressive given the subjectivity — frame it honestly

Task L5: Help With Presentation Content

[HIGH] Implementation Steps

- Write the “Data Collection” slide content: what was scraped, how many records, data quality
- Write the “Problem Statement” slide: why the old heuristic was circular, why real labels matter
- Provide 2-3 interesting stats from the data (e.g., “12% of UFC fights receive a bonus”)

GUSTAVE — Features — Cross-Features & Code Quality

You own the feature pipeline and code quality. The 24 matchup cross-features are already implemented but never connected — your main job is to wire them in. Then clean up every piece of dead code so the repo looks professional.

Day 1 Tasks

Task G1: Activate the 24 Cross-Features

This is the single fastest high-impact fix. The features are ALREADY CODED in extract_matchup_features(). They are just never concatenated into the input vector.

[CRITICAL] Implementation Steps

- Open models/feature_engineering.py, find build_matchup_vector()
- Currently it only concatenates fighter_a_features + fighter_b_features (24+24=48)
- Change it to: fighter_a_features + fighter_b_features + matchup_features (24+24+24=72)
- Call extract_matchup_features(fighter_a_stats, fighter_b_stats) to get the 24 cross-features
- Update the config: set input_dim = 72 (was 48)
- Update the StandardScaler — it must be re-fitted on the 72-dim vectors
- Test: print the shape of one feature vector to confirm it's (72,)
- IMPORTANT: coordinate with Mattheus — his split code needs to know the new feature dimension

Task G2: Fix Dead Code and Bugs

Clean these up so the codebase looks professional for submission.

[HIGH] Implementation Steps

- ResidualBlock: either wire it into FightQualityNN.forward() or DELETE the class entirely. If dataset < 5k fights, delete it — residual connections don't help small MLPs
- Fix --heuristic flag in main.py (lines 217-222): the bare 'pass' makes HeuristicMatchmaker dead code. Either implement the heuristic path properly or remove the flag
- Remove unused import: UFCApiWrapper in pipeline.py
- Fix parse_stat type annotation in ufc_stats_scraper.py: annotated as -> int but returns tuple
- Remove python-dotenv from requirements.txt (nothing calls load_dotenv())
- Fix Kaiming init: change nonlinearity='relu' to nonlinearity='leaky_relu' (closest to GELU) or switch activation to ReLU for consistency
- Fix README: remove references to sherdog_scraper.py and data/schema.sql (they don't exist)

Task G3: PCA Dimensionality Analysis (Optional but Impressive)

With 72 features, it's worth checking if there's redundancy. This shows the professor you understand dimensionality reduction.

[HIGH] Implementation Steps

- from sklearn.decomposition import PCA
- Fit PCA on the 72-feature training set (after scaling)
- Plot cumulative explained variance ratio — how many components capture 95% of variance?
- If 30-40 components capture 95%+, note this in the presentation as evidence of feature redundancy
- Do NOT replace the original features with PCA components for the main model — just do this as an analysis
- Save the variance plot as a PNG for the slides

Day 2 Tasks

Task G4: Update Dashboard

[HIGH] Implementation Steps

- Update the Rich terminal dashboard to show the new model's predictions (binary probability, not 0–100 score)
- Add a column for "Bonus Fight Probability" in matchup displays
- Ensure the star rating system still works (map probability to 1–5 stars: >0.8 = 5 stars, >0.6 = 4, etc.)
- Test the CLI commands: predict, matchup, card — verify they run without errors

Task G5: Clean Up CLI

[HIGH] Implementation Steps

- Update the 'evaluate' command to use the new temporal test set (not the full dataset)
- Add a 'compare' command that runs all baselines and prints the comparison table
- Ensure 'demo' mode still works with synthetic fighters for quick testing

MATTHEUS — Evaluation — Splits, CV & Presentation

You own the evaluation methodology and the final presentation. Your temporal split is what makes the evaluation scientifically valid. Your slides are what makes the grade.

Day 1 Tasks

Task M1: Implement Temporal Train/Val/Test Split

This is the most important methodological fix. Random splitting lets the model see future fighter stats when predicting past fights, which is data leakage.

[CRITICAL] Implementation Steps

- Create models/data_splits.py with a function temporal_split(fights_df, val_cutoff, test_cutoff)
- Default cutoffs: train = all fights before 2023-01-01, val = 2023-01-01 to 2023-12-31, test = 2024-01-01+
- Sort all fights by event_date before splitting
- Return: X_train, y_train, X_val, y_val, X_test, y_test
- Print split sizes: e.g., “Train: 3500 fights, Val: 400 fights, Test: 300 fights”
- CRITICAL: verify no temporal overlap — assert max(train_dates) < min(val_dates) < min(test_dates)

Task M2: Fix Symmetric Augmentation Leakage

Currently, both (A,B) and (B,A) orderings of a fight can end up in different splits. This inflates validation metrics because the model has effectively seen the fight.

[CRITICAL] Implementation Steps

- In the augmentation step, assign a fight_id to each pair
- Both (A,B) and (B,A) must share the same fight_id
- Split by fight_id, NOT by individual rows — use GroupShuffleSplit or manual grouping
- After temporal split, THEN augment within each split separately
- The correct order is: split by date → augment within each split → never cross-contaminate

Task M3: Stratified K-Fold Cross-Validation

Within the training set, use k-fold CV for hyperparameter tuning. This gives more robust estimates than a single split.

[HIGH] Implementation Steps

- Use TimeSeriesSplit(n_splits=5) from sklearn — this respects temporal ordering within the training set

- For tree-based models (XGBoost, RF): use this inside RandomizedSearchCV directly
- For the NN: implement a manual k-fold loop since PyTorch doesn't integrate with sklearn CV natively
- Report mean ± std of F1 across folds for each model — this goes in the presentation
- Alternative: StratifiedKFold(n_splits=5) if TimeSeriesSplit gives too-small folds (check fold sizes first)

Day 2 Tasks

Task M4: Calibration Analysis

When the model predicts 70% probability of a bonus fight, is it right 70% of the time? Calibration answers this.

[HIGH] Implementation Steps

- from sklearn.calibration import calibration_curve, CalibrationDisplay
- Compute calibration curves for all models on the test set
- Plot: x-axis = predicted probability, y-axis = actual fraction of bonus fights
- A well-calibrated model follows the diagonal. If it's way off, note that the probabilities are not trustworthy as absolute values but ranking is still valid
- Save plot as PNG for the slides

Task M5: Spearman Rank Correlation

For a matchmaker, the ranking of fights matters more than exact probability values.

[HIGH] Implementation Steps

- from scipy.stats import spearmanr
- For each model, compute Spearman's rho between predicted probabilities and actual labels on the test set
- Also compute on a per-event basis: within each UFC card, does the model rank the bonus fight(s) higher than non-bonus fights?
- Report: overall Spearman's rho, per-event mean rank of actual bonus fights

Task M6: Build the Presentation Slides

You own the slide deck. Target 12–15 slides. Coordinate with everyone to get their outputs.

[CRITICAL] Slide Structure

- Slide 1: Title + Team Members
- Slide 2: Problem Statement — "What makes a great UFC fight?"
- Slide 3: The Circular Training Problem (why the old approach was wrong)
- Slide 4: Our Solution — real labels, proper methodology

- Slide 5: Data Collection — sources, volume, quality (Lorenzo's content)
- Slide 6: Feature Engineering — 48 → 72 features, explain cross-features (Gustave's content)
- Slide 7: PCA analysis plot (Gustave's output)
- Slide 8: Training Methodology — temporal split, k-fold CV, class imbalance handling
- Slide 9: Model Comparison Table — all baselines vs NN (Raji's output)
- Slide 10: SHAP Feature Importance (Raji's plots)
- Slide 11: Calibration + Rank Correlation plots
- Slide 12: Backtesting Results — "Did the model predict real FOTN winners?" (Lorenzo's output)
- Slide 13: Matchmaker Demo — show predictions for an upcoming card
- Slide 14: Lessons Learned + Future Work
- Slide 15: Q&A

Coordination Rules

Git Workflow

Branch naming: Each person works on their own branch: raji/baselines, lorenzo/labels, gustave/features, mattheus/splits

Merge order matters: Lorenzo's labels FIRST → Gustave's features SECOND → Mattheus's splits THIRD → Raji integrates LAST

Commit often: Small commits with descriptive messages. Push at least every 2 hours so others can see progress.

No force pushes: Ever. If you mess up, ask the team before rewriting history.

Dependencies Between Tasks

Task	Depends On	Blocks
Lorenzo: Labels	Nothing (start immediately)	Everyone else
Gustave: Cross-features	Nothing (start immediately)	Raji, Mattheus
Mattheus: Temporal split	Lorenzo's labels	Raji's baselines
Raji: XGBoost/RF	All three above	Final comparison
Raji: NN tuning	All three above	Final comparison
Lorenzo: Backtesting	Raji's trained models	Presentation
Mattheus: Calibration	Raji's trained models	Presentation
Gustave: Dashboard	Raji's integration	Demo

Communication

Sync checkpoints: Quick standup at hour 6 (end of Day 1 morning), hour 12 (end of Day 1), and hour 20 (Day 2 afternoon).

Blockers: If you are blocked, message the team chat IMMEDIATELY. Do not wait for a standup.

Done = pushed + tested: A task is not done until your code is pushed to your branch and you have verified it runs without errors.

Key Technical Reminders

Class imbalance: FOTN fights are ~10-15% of all fights. Every model must handle this: scale_pos_weight for XGBoost, class_weight='balanced' for RF/LR, weighted BCELoss for the NN.

Feature scaling: StandardScaler must be fit ONLY on training data, then transform val and test. Never fit on the full dataset.

No test set peeking: The test set (2024+ fights) is touched ONCE at the very end for final metrics. If you are tuning on test set metrics, you are doing it wrong.

Reproducibility: Set random seeds everywhere: random_state=42 for sklearn, torch.manual_seed(42), np.random.seed(42).

What "Good" Looks Like

A strong submission will show that the team identified the circular training problem, replaced it with real labels, applied proper ML methodology (temporal splits, cross-validation, hyperparameter tuning, multiple baselines), and communicated the results honestly with interpretability tools (SHAP, calibration). If XGBoost beats the neural network, that is a perfectly valid result — the intellectual honesty and methodology matter more than the model architecture.